

Below is the E2 completion doc in text form. You can drop this into a markdown file (e.g. docs/phaseE2/PhaseE2_Completion.md) and export to PDF.

Slice – E2 Completion Document

Module: Thesis → Quant → Risk/Scenario Evaluation Path

Phase: E2

Status: Complete

1. Objective

E2's purpose is to implement a **deterministic, non-LLM evaluation pipeline** for a single Thesis object, turning its expression legs into:

- A constant-weight portfolio constructed from the legs,
- A historical equity curve over an inferred backtest window,
- Performance and risk metrics,
- Simple shock scenarios,

and to expose this via a clean service API that can be called from higher layers (context builder, UI) without coupling to any LLM or UI code.

This evaluation path must be:

- **Pure Python + pandas** (no HTTP/LLM/UI),
 - **Deterministic** (no randomness or hidden state),
 - **Test-covered** against a concrete checklist of behaviors.
-

2. Scope of E2

E2 **includes**:

- A minimal but explicit PriceSource abstraction.
- DTOs to represent evaluation outputs.
- A ThesisEvaluationService implementing the full evaluation pipeline.
- A focused unit test suite validating implementation against the E2 spec.

E2 **does NOT** include:

- Any Streamlit/HTTP/UI wiring.
 - Any LLM-based critique, explanation, or narrative.
 - Multi-thesis or portfolio-of-theses aggregation.
 - Backtrader-based strategy logic (we do not run strategies here – we operate off daily prices via PriceSource).
-

3. Implemented Artifacts

3.1

src/slice/quant/price_source.py

Purpose: Abstraction for historical and current prices, isolating evaluation logic from any specific data backend.

Implementation:

- Defines a protocol-like interface:

```
from datetime import date
from typing import Protocol

import pandas as pd
```

```

class PriceSource(Protocol):
    def get_history(self, asset: str, start: date, end: date) -> pd.Series:
        """
        Daily prices for asset between start and end.

        Requirements:
        - Index: datetime/date, monotonic ascending.
        - Values: float closing prices.
        """
        ...

    def get_current_price(self, asset: str) -> float:
        """
        Latest available price for asset.
        """
        ...

```

Notes:

- No concrete implementation is defined in E2 – the abstraction is sufficient.
- Existing infra (e.g. DB loaders, quant_engine) can later be wrapped to satisfy this interface.

3.2

src/slice/models/evaluation.py

Purpose: DTOs for deterministic evaluation outputs.

Implementation:

- EquityPoint – single point on the equity curve:

```

class EquityPoint(BaseModel):
    date: datetime
    value: float

```

-

- ScenarioImpact – P&L impact of a defined scenario:

```
class ScenarioImpact(BaseModel):
    name: str
    pnl_abs: float # absolute P&L in equity units
    pnl_pct: float # % P&L, e.g. -10.0 = -10.0%
```

- ThesisEvaluationResult – top-level evaluation result:

```
class ThesisEvaluationResult(BaseModel):
    performance: Dict[str, float]
    timeseries: List[EquityPoint]
    risk_metrics: Dict[str, float]
    scenarios: List[ScenarioImpact]
```

Contract (stable keys):

- performance keys:
 - total_return
 - cagr
 - volatility
 - sharpe
 - max_drawdown
- risk_metrics keys:
 - max_weight_pct
 - VaR_95
 - max_drawdown_pct
- scenarios: list of ScenarioImpact with at least:
 - "All -10%"
 - "All +10%"

3.3

src/slice/models/__init__.py

Purpose: Expose evaluation DTOs at the slice.models package level.

Implementation:

```
from .evaluation import EquityPoint, ScenarioImpact, ThesisEvaluationResult
```

No behavioral logic; this is purely an import surface for other modules.

3.4

src/slice/evaluation/thesis_evaluation.py

Purpose: Implement the core E2 evaluation pipeline.

Key type:

```
@dataclass
class ThesisEvaluationService:
    price_source: PriceSource
```

3.4.1 Validation

Method: `_validate_thesis_expression(self, thesis: Thesis) -> List[ThesisExpressionLeg]`

Checks:

- `thesis.expression` is a non-empty list (if you manage to construct a Thesis with an empty list, Pydantic will already reject it; the service still guards its own assumptions).
- Each leg:
 - `direction` is `Direction.LONG` or `Direction.SHORT`.
 - `size_pct` is not `None` and `>= 0`.
- Sum of `size_pct` across legs must satisfy:

```
sum(size_pct) ≤ 100.0 (allowing small epsilon for float error)
```

On violation, raises `ValueError` with a clear message (e.g. “exceeds 100%”, “Invalid direction for asset A”).

3.4.2 Window Determination

Method: `_determine_backtest_window(self, thesis: Thesis) -> tuple[date, date]`

Logic:

- Attempts to parse:

```
start = date.fromisoformat(thesis.start_date)    # if present/valid
end = date.fromisoformat(thesis.review_date)     # if present/valid
```

-

- Fallbacks:
 - If start is missing/invalid: start = today - 252*2 days (approx 2 trading years).
 - If end is missing/invalid or end <= start: end = today.

Always returns (start, end) with end > start.

3.4.3 Price Loading

Method: `_load_price_history(self, legs, start, end) -> pd.DataFrame`

Logic:

- For each leg, calls `self.price_source.get_history(leg.asset, start, end)`:
 - If the returned series is None or .empty, raises:

```
ValueError("No price data for asset {asset}.")
```

-

- Builds:

```
prices_df = pd.DataFrame(series_by_asset)    # columns = asset tickers
prices_df = prices_df.dropna(how="any")      # strict overlap
```

-

- If `prices_df` is .empty after alignment, raises:

```
ValueError("No overlapping price history for thesis assets.")
```

3.4.4 Weights

Method: `_build_weights(self, legs) -> dict[str, float]`

Logic:

- For each leg:

```
direction_sign = +1.0 for LONG, -1.0 for SHORT  
weight[asset] = (size_pct / 100.0) * direction_sign
```

-
- No renormalization if $\text{sum}(\text{weights}) < 1.0$. Remaining capital is treated as cash at 0% return.

3.4.5 Returns and Equity Curve

In `evaluate_thesis`:

- Compute daily returns for each asset:

```
daily_returns_df = prices_df.pct_change().fillna(0.0)
```

-
- Portfolio returns:

```
weight_series = pd.Series(weights)  
daily_portfolio_returns = daily_returns_df.mul(weight_series,  
axis=1).sum(axis=1)
```

-
- Equity curve (starting at 1.0):

```
equity_curve = (1.0 + daily_portfolio_returns).cumprod()
```

3.4.6 Performance Metrics

Method: `_compute_performance_metrics(equity_curve, daily_returns)` -> dict[str, float]

- If equity_curve is empty or invalid, returns zeros for all metrics.
- Otherwise:
 - total_return:

```
total_return_frac = final_value / initial_value - 1.0
total_return = total_return_frac * 100.0
```

- -
 - years = len(equity_curve) / 252.0
 - cagr:

```
cagr_frac = (final_value / initial_value) ** (1.0 / years) - 1.0 # if
years > 0
cagr = cagr_frac * 100.0
```

- -
 - volatility:

```
vol_daily = daily_returns.std()
vol_annual = vol_daily * sqrt(252)
volatility = vol_annual * 100.0
```

- -
 - sharpe:

```
sharpe = cagr_frac / vol_annual if vol_annual > 0 else 0.0
```

- -
 - max_drawdown:

```
running_max = equity_curve.cummax()
drawdowns = equity_curve / running_max - 1.0
max_dd_frac = -drawdowns.min()
max_drawdown = max_dd_frac * 100.0
```

3.4.7 Risk Metrics

Method: `_compute_risk_metrics(weights, daily_returns, max_drawdown_frac) -> dict[str, float]`

- `max_weight_pct`: `max(abs(w) for w in weights.values()) * 100.0`
- `VaR_95`:
 - If empty returns: 0.0.
 - Else:

```
q = daily_returns.quantile(0.05)      # 5% quantile
VaR_95 = -q * 100.0                   # positive % loss magnitude
```

-
- `max_drawdown_pct`: `max_drawdown_frac * 100.0`

3.4.8 Scenario Impacts

Method: `_compute_scenarios(prices_df, weights, equity_curve) -> List[ScenarioImpact]`

- Uses the last row of `prices_df` as the current price vector.
- Base equity is the final value of `equity_curve`.
- For a given new price set:

```
asset_return = (new_price - old_price) / old_price
pnl_frac += weight[asset] * asset_return
```

-
- Scenarios implemented:
 1. "All -10%": `new_price = old_price * 0.9` for all assets.
 2. "All +10%": `new_price = old_price * 1.1` for all assets.
- For each scenario:

```
pnl_pct = pnl_frac * 100.0
pnl_abs = pnl_frac * final_equity
```

Outputs `ScenarioImpact(name, pnl_abs, pnl_pct)`.

3.4.9 Public Entry Point

`evaluate_thesis(self, thesis: Thesis) -> ThesisEvaluationResult` orchestrates:

1. `_validate_thesis_expression`
2. `_determine_backtest_window`
3. `_load_price_history`
4. Daily returns + weights + portfolio returns
5. `equity_curve`
6. `performance = _compute_performance_metrics(...)`
7. `risk_metrics = _compute_risk_metrics(...)`
8. `scenarios = _compute_scenarios(...)`
9. Build timeseries: `List[EquityPoint]` from `equity_curve` index and values.

Returns a fully-populated `ThesisEvaluationResult`.

4. Test Suite

File: `tests/evaluation/test_thesis_evaluation.py`

4.1 FakePriceSource

A minimal, in-memory `PriceSource` used in all tests:

- `__init__(series_by_asset: Dict[str, pd.Series])`
- `get_history(asset, start, end)` returns stored series or empty.
- `get_current_price(asset)` returns the last value.

4.2 Helper

`_make_basic_thesis_single_leg(...)` constructs a valid, single-leg Thesis with correct drivers, disconfirmers, etc., preserving compatibility with the existing Thesis model.

4.3 Behavior Tests

Covered cases:

1. **Single asset, monotonic up** – test_single_asset_monotonic_up
 - 100% long, strictly increasing prices.
 - Asserts:
 - total_return > 0
 - max_drawdown ≈ 0
 - max_weight_pct == 100
 - Timeseries length equals price series length.
 - Initial equity ≈ 1.0 .
2. **Single asset, monotonic down** – test_single_asset_monotonic_down
 - 100% long, strictly decreasing prices.
 - Asserts:
 - total_return < 0
 - max_drawdown $\approx$ |total_return| (no recovery).
3. **Two assets, offsetting returns** – test_two_assets_offsetting_returns
 - 50% long A (up), 50% long B (down), symmetric moves.
 - Asserts:
 - total_return ≈ 0
 - max_weight_pct == 50.
4. **Short position** – test_single_asset_short_position
 - 100% short an asset that rises.
 - Asserts:
 - total_return < 0
 - Scenarios exist: "All -10%", "All +10%"
 - For a 100% short:
 - "All -10%": pnl_pct $\approx +10$
 - "All +10%": pnl_pct ≈ -10 .
5. **Partial cash** – test_partial_cash_allocation
 - 50% long in a doubling asset, 50% cash.
 - Asserts:
 - total_return ≈ 50
 - max_weight_pct == 50.
6. **Scenario presence** – test_scenarios_all_plus_minus_10_present
 - Simple 100% long thesis.
 - Asserts:
 - Scenarios set contains "All -10%" and "All +10%".
 - len(result.scenarios) ≥ 2 .
7. **VaR_95 correctness** – test_var_95_for_constant_minus_10_returns
 - Constant daily return of -10%.
 - Asserts:
 - VaR_95 ≈ 10.0 .

8. **Risk max_drawdown_pct alignment** –
`test_risk_max_drawdown_pct_matches_performance`
 - Engineered equity path with exact 25% drawdown.
 - Asserts:
 - `performance["max_drawdown"] ≈ 25`
 - `risk["max_drawdown_pct"] ≈ 25`.
9. **Scenario pnl_abs correctness** – `test_scenario_pnl_abs_uses_final_equity`
 - Flat price path → final equity ≈ 1.0 , zero realized P&L.
 - For "All +10%":
 - `pnl_pct ≈ 10`
 - `pnl_abs ≈ 0.10 * final_equity`.

4.4 Error / Validation Tests

1. **Weights exceed 100%** – `test_weights_exceed_100_raises`
 - Legs with total size_pct = 110.
 - Asserts:
 - `ValueError` raised with message containing "exceeds 100%".
 2. **Empty expression** – `test_empty_expression_raises`
 - Attempts to construct a Thesis with `expression=[]`.
 - Asserts:
 - `Pydantic ValidationError` and error text "expression cannot be empty".
 3. **Missing price data** – `test_missing_price_data_raises`
 - `PriceSource` only provides series for one asset.
 - Asserts:
 - `ValueError` raised with "No price data for asset B".
 4. **Invalid direction** – `test_invalid_direction_raises_value_error_from_service`
 - Uses `.construct()` to bypass `Pydantic` and inject a leg with `direction="INVALID"`.
 - Asserts:
 - `ThesisEvaluationService.evaluate_thesis(...)` raises `ValueError` containing "Invalid direction for asset A".
-

5. Independence from LLM/UI

- `src/slice/evaluation/thesis_evaluation.py` imports:
 - `dataclasses`, `datetime`, `typing`, `pandas`,
 - `slice.models.*`, `slice.quant.price_source`.
- It does **not** import:
 - Any LLM libraries (`openai`, etc.),
 - Any HTTP/client libraries (`requests`, `httpx`, etc.),
 - Any UI frameworks (`streamlit`, `fastapi`, etc.).

The E2 path is strictly a **pure computation module**, suitable for use by any caller (context builder, CLI, UI, LLM, etc.) without reverse dependency.

6. Integration Notes & Assumptions

- The evaluation service is **not yet wired** into DataAccess or any UI layer. Integration is expected in a later phase (E4/E6 or similar).
 - PriceSource is currently only an interface; production code must provide a concrete implementation (e.g. wrapping quant_engine loaders, a DB layer, or an API).
 - All percentages (total_return, cagr, volatility, max_drawdown, max_drawdown_pct, VaR_95, pnl_pct) are expressed in **percent units**, not decimals.
 - Time scaling assumes **252 trading days/year**, consistent with the rest of the quant stack.
-

7. E2 Completion Checklist

Artifacts

- PriceSource interface → **PASS**
- Evaluation DTOs (EquityPoint, ScenarioImpact, ThesisEvaluationResult) → **PASS**
- ThesisEvaluationService with required behavior → **PASS**

Metrics & Scenarios

- Equity curve, performance metrics, risk metrics, scenarios implemented as specified → **PASS**

Tests

- All required behavior and error cases covered by unit tests, including:
 - Performance/risk behavior in multiple regimes,
 - VaR_95 and max_drawdown_pct correctness,
 - Scenario pnl_pct and pnl_abs correctness,
 - Validation / error conditions → **PASS**

Independence from LLM/UI

- No prohibited imports or dependencies → **PASS**

Overall E2 Status: COMPLETE (per spec and codified checklist).